Universidad de La Habana
Facultad de Matemática y Computación

Diseño e implementación de una plataforma informática para el autorreporte y la gestión operativa de eventos adversos post-vacunación en el Instituto Finlay de Vacunas

Autor:
Jabel Resendiz Aguirre

Tutores:

MsC. Joanna Campbell Amos
MsC. Aracelys García Armenteros

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

22 de abril de 2026

github.com/JabelResendiz/thesis

Dedicación

Agradecimientos

Agradecimientos

Opinión del tutor

Opiniones de los tutores

Resumen

Resumen en español

Abstract

Resumen en inglés

Índice general

1. Estado del Arte **1**
1.1. Análisis de sistemas de autorreporte clínico existentes 1
1.1.1. Vaccine Adverse Event Reporting System (VAERS) 1
1.1.2. VAERS (Vaccine Adverse Event Reporting System) 4
1.1.3. Sistema NotificaRAM de la Agencia Española de Medicamentos y Productos Sanitarios 4
1.1.4. Sistema V-safe del Centers for Disease Control and Prevention 6
1.1.5. Análisis comparativo de los sistemas estudiados 8
1.2. Análisis de tecnologías para la implementación del sistema propuesto 8
1.2.1. Criterios técnicos para la selección de tecnologías 9
1.2.2. Tecnologías para el desarrollo del backend 9
1.2.3. Conclusiones sobre las plataformas de desarrollo expuestas . . 12
1.2.4. Tecnologías para el desarrollo del frontend 16
1.2.5. Conclusiones sobre los frameworks de frontend expuestos . . . 18
1.2.6. Sistemas de gestión de bases de datos 20
1.2.7. Conclusiones sobre los sistemas de gestión de bases de datos expuestos 21
1.2.8. Síntesis comparativa de las tecnologías analizadas 21
1.3. Tecnologías y enfoques para la gestión de datos clínicos 23
1.3.1. Estándares de interoperabilidad clínica 23
1.3.2. Modelos de captura de datos clínicos 25
1.3.3. Herramientas para la validación reactiva de formularios 26
1.3.4. Calidad e integración de datos clínicos 26
1.4. Conclusiones del capítulo 27

2. Propuesta **29**

3. Detalles de Implementación y Experimentos **30**

Conclusiones **31**

Recomendaciones **32**

A. Formularios de Referencia **33**
A.1. Formulario VAERS - Octubre 2025 33

Bibliografía **35**

Índice de figuras

1.1. Logotipo del sistema VAERS (vaers.hhs.gov) 2
1.2. Ejemplo del formulario en formato PDF editable utilizado para el reporte de eventos adversos en el sistema VAERS 3
1.3. Ejemplo del formulario web utilizado para el reporte de eventos adversos en el sistema VAERS 3
1.4. Logotipo del sistema NotificaRAM (notificaram.es) 5
1.5. Logotipo del sistema V-safe (vsafe.cdc.gov) 6
1.6. Ejemplo del flujo de interacción del sistema V-safe mediante encuestas digitales 7
1.7. Logo de .NET. 10
1.8. Logo de Spring Boot 11
1.9. Logo de NestJS 11
1.10. Throughput (respuestas por segundo) en .NET, Spring Boot y Node.js (adaptado de TechEmpower Framework Benchmarks) 13
1.11. Prueba de estrés sobre operaciones POST: uso de memoria y CPU (adaptado de *Performance Comparison of RESTful Web APIs using a Test Suite: .NET vs. Java Spring Boot* [18]) 14
1.12. Logo de React 16
1.13. Funcionamiento del Virtual DOM en React (adaptado de *Virtual DOM in React JS*, GeeksforGeeks [24]) 17
1.14. Logo de Angular 17
1.15. Logo de Vue.js 18
1.16. Ejemplos de sistemas de gestión de bases de datos relacionales y NoSQL 20
1.17. Estructura jerárquica del sistema de codificación MedDRA. 24

A.1. Formulario VAERS - Octubre 2025 34

Ejemplos de código

Capítulo 1

Estado del Arte

El presente capítulo tiene como objetivo analizar los fundamentos teóricos, tecnológicos y los sistemas existentes relacionados con la farmacovigilancia digital y el desarrollo de plataformas de autorreporte clínico. Se aborda el estudio de soluciones implementadas a nivel internacional, así como las tecnologías y estándares utilizados en sistemas de salud digital, con el propósito de establecer una base sólida que sustente el diseño e implementación del sistema propuesto. Asimismo, se revisan conceptos relacionados con el modelado de datos clínicos y estándares de interoperabilidad que garantizan la consistencia y calidad de la información.

1.1. Análisis de sistemas de autorreporte clínico existentes

En el contexto de la farmacovigilancia moderna, diversos sistemas han sido desarrollados con el objetivo de facilitar la notificación de eventos adversos posteriores a la vacunación. Estos sistemas difieren en su enfoque, arquitectura y nivel de digitalización, lo que impacta directamente en la calidad y disponibilidad de la información recolectada. A continuación, se analizan algunas de las principales soluciones implementadas a nivel internacional.

1.1.1. Vaccine Adverse Event Reporting System (VAERS)

VAERS Vaccine Adverse Event Reporting System
www.vaers.hhs.gov

Figura 1.1: Logotipo del sistema VAERS (vaers.hhs.gov)

El *Vaccine Adverse Event Reporting System* (VAERS) [1] es un sistema nacional de vigilancia de seguridad de vacunas en los Estados Unidos, diseñado para el monitoreo de eventos adversos posteriores a la inmunización [2]. Este sistema se enmarca en el proceso de supervisión de vacunas tras su aprobación por la *Food and Drug Administration* (FDA) [3], y es administrado conjuntamente por la FDA y los *Centers for Disease Control and Prevention* (CDC) [2, 3], y funciona como un repositorio centralizado para la recolección y análisis de reportes de eventos adversos asociados a la vacunación.

VAERS se clasifica como un sistema de vigilancia pasiva, ya que depende de la notificación voluntaria de eventos adversos por parte de los usuarios, en lugar de la recolección activa de datos por parte de las instituciones de salud [2]. Este enfoque permite una amplia cobertura poblacional, aunque introduce limitaciones asociadas a la calidad y completitud de los reportes.

El sistema permite la notificación de eventos adversos por parte de diversos actores, incluyendo profesionales de la salud, fabricantes de vacunas, pacientes y familiares [2, 3]. En determinados casos, la legislación estadounidense establece la obligatoriedad de reportar ciertos eventos adversos, particularmente para proveedores de salud y fabricantes. Asimismo, con el objetivo de garantizar la veracidad y confiabilidad de los datos, el sistema contempla disposiciones legales que sancionan el envío intencional de información falsa, considerándose una violación de la legislación federal de los Estados Unidos [1].

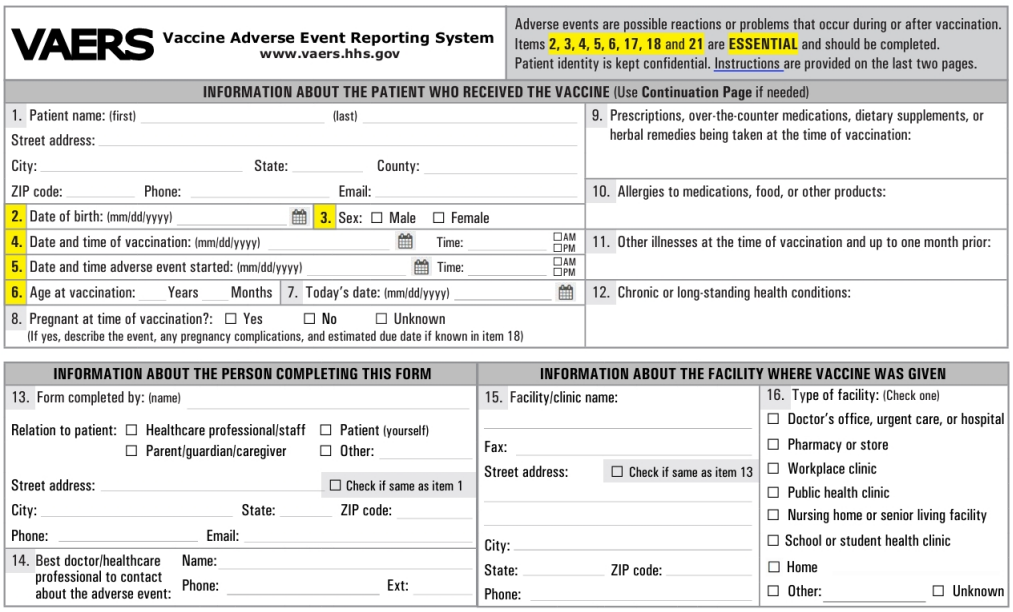
El reporte de eventos adversos se realiza principalmente a través de plataformas digitales accesibles vía web [1], donde los usuarios pueden completar formularios estructurados que son posteriormente almacenados en una base de datos centralizada. Adicionalmente, el sistema contempla la posibilidad de notificación mediante formularios en formato PDF descargable, los cuales pueden ser completados de manera manual y posteriormente remitidos a través de la plataforma oficial.

En las Figuras 1.2 y 1.3 se muestran ejemplos representativos de ambos mecanismos de notificación, correspondientes al formulario en PDF y al formulario web respectivamente.

Este mecanismo facilita el acceso al sistema y permite la recopilación masiva de información a nivel nacional.

Entre las principales fortalezas de VAERS se encuentra su capacidad para actuar como un sistema de alerta temprana en la detección de eventos adversos poco frecuentes o inesperados. Asimismo, su carácter abierto permite la participación de múltiples actores, incrementando la cantidad de información disponible para el análisis [2]. Adicionalmente, los datos recopilados son accesibles públicamente, lo que contribuye a la transparencia en la vigilancia de la seguridad vacunal.

No obstante, VAERS presenta limitaciones inherentes a los sistemas de vigilancia pasiva. Entre ellas se encuentran el subregistro de eventos, la variabilidad en la calidad



VAERS Vaccine Adverse Event Reporting System
www.vaers.hhs.gov

Adverse events are possible reactions or problems that occur during or after vaccination. Items 2, 3, 4, 5, 6, 17, 18 and 21 are ESSENTIAL and should be completed. Patient identity is kept confidential. Instructions are provided on the last two pages.

INFORMATION ABOUT THE PATIENT WHO RECEIVED THE VACCINE (Use Continuation Page if needed)

1. Patient name: (first) (last)
Street address:
City: State: County:
ZIP code: Phone: Email:
2. Date of birth: (mm/dd/yyyy) 3. Sex: ☐ Male ☐ Female
4. Date and time of vaccination: (mm/dd/yyyy) Time: ☐AM ☐PM
5. Date and time adverse event started: (mm/dd/yyyy) Time: ☐AM ☐PM
6. Age at vaccination: Years Months 7. Today's date: (mm/dd/yyyy)
8. Pregnant at time of vaccination?: ☐ Yes ☐ No ☐ Unknown
(If yes, describe the event, any pregnancy complications, and estimated due date if known in item 18)
9. Prescriptions, over-the-counter medications, dietary supplements, or herbal remedies being taken at the time of vaccination:
10. Allergies to medications, food, or other products:
11. Other illnesses at the time of vaccination and up to one month prior:
12. Chronic or long-standing health conditions:

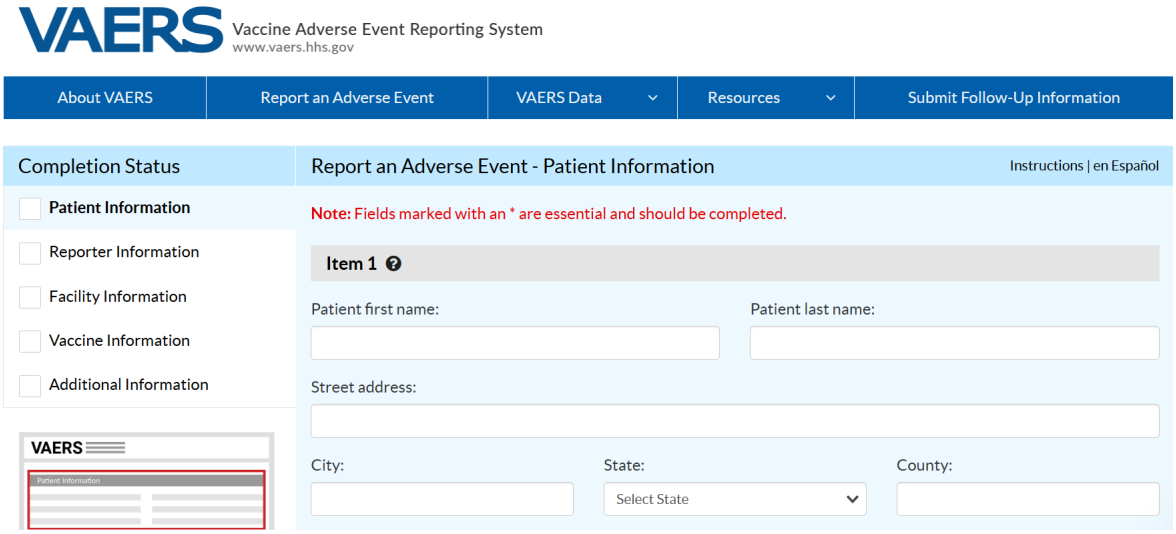
INFORMATION ABOUT THE PERSON COMPLETING THIS FORM

13. Form completed by: (name)
Relation to patient: ☐ Healthcare professional/staff ☐ Patient (yourself) ☐ Parent/guardian/caregiver ☐ Other:
Street address: ☐ Check if same as item 1
City: State: ZIP code:
Phone: Email:
14. Best doctor/healthcare professional to contact about the adverse event: Name: Phone: Ext:

INFORMATION ABOUT THE FACILITY WHERE VACCINE WAS GIVEN

15. Facility/clinic name:
Fax:
Street address: ☐ Check if same as item 13
City:
State: ZIP code:
Phone:
16. Type of facility: (Check one)
☐ Doctor's office, urgent care, or hospital
☐ Pharmacy or store
☐ Workplace clinic
☐ Public health clinic
☐ Nursing home or senior living facility
☐ School or student health clinic
☐ Home
☐ Other: ☐ Unknown

Figura 1.2: Ejemplo del formulario en formato PDF editable utilizado para el reporte de eventos adversos en el sistema VAERS



VAERS Vaccine Adverse Event Reporting System
www.vaers.hhs.gov

About VAERS | Report an Adverse Event | VAERS Data | Resources | Submit Follow-Up Information

Completion Status
- Patient Information
- Reporter Information
- Facility Information
- Vaccine Information
- Additional Information

Report an Adverse Event - Patient Information
Instructions | en Español

Note: Fields marked with an * are essential and should be completed.

Item 1

Patient first name: Patient last name:
Street address:
City: State: Select State County:

Figura 1.3: Ejemplo del formulario web utilizado para el reporte de eventos adversos en el sistema VAERS

de los datos reportados y la posible existencia de sesgos de notificación [2]. Además, los reportes no permiten establecer relaciones causales directas entre la vacuna y el evento adverso, ya que muchos de estos pueden ocurrir de manera coincidente tras la inmunización [3]. En este sentido «Tras realizar investigaciones, los científicos concluyen que la mayoría de los eventos notificados al VAERS no están asociados con la vacunación» (traducción propia) [2]. Estas limitaciones requieren que los posibles riesgos identificados sean evaluados mediante sistemas complementarios y estudios más rigurosos, evidenciando la necesidad de enfoques más robustos en el diseño de sistemas de farmacovigilancia.

1.1.2. Sistema NotificaRAM de la Agencia Española de Medicamentos y Productos Sanitarios

Figura 1.4: Logotipo del sistema NotificaRAM (notificaram.es)

El sistema **notificaRAM** constituye el formulario electrónico oficial de notificación de sospechas de reacciones adversas a medicamentos en España, permitiendo la comunicación de posibles efectos adversos por parte de ciudadanos y profesionales sanitarios [4]. Este formulario se integra en el Sistema Español de Farmacovigilancia de Medicamentos de Uso Humano (SEFV-H), una red nacional coordinada por la Agencia Española de Medicamentos y Productos Sanitarios (AEMPS) para la recogida, registro y evaluación de sospechas de reacciones adversas [5].

El sistema se basa en un modelo de notificación espontánea, en el que las sospechas de reacciones adversas pueden ser comunicadas tanto por profesionales sanitarios como por la ciudadanía [6]. Los profesionales sanitarios deben notificar estas sospechas cuando las detectan, mientras que la ciudadanía puede hacerlo cuando sospecha una posible reacción adversa [6].

Los actores implicados en la notificación incluyen profesionales sanitarios (como médicos, farmacéuticos y personal de enfermería) y ciudadanos o pacientes. Ambos grupos pueden comunicar sospechas de reacciones adversas mediante los formularios disponibles en la plataforma [4, 6].

La notificación se realiza a través de formularios electrónicos disponibles en la plataforma oficial `https://www.notificaram.es`, donde el usuario selecciona el tipo de formulario según sea ciudadano o profesional sanitario. Las notificaciones son remitidas a los centros autonómicos de farmacovigilancia para su evaluación e incorporación al sistema [4, 5].

El SEFV-H es una red descentralizada en la que los centros autonómicos de farmacovigilancia registran y evalúan las sospechas de reacciones adversas en una base de datos común denominada **FEDRA** (Farmacovigilancia Española, Datos de Reacciones Adversas) [7]. Este sistema permite la recogida y análisis coordinado de las notificaciones dentro del sistema nacional de farmacovigilancia [5].

Asimismo, los casos registrados en FEDRA son transmitidos a la base de datos europea de sospechas de reacciones adversas **EudraVigilance**, utilizada por las agencias reguladoras de la Unión Europea para la detección de señales de seguridad [6].

El sistema presenta limitaciones propias del modelo de notificación espontánea, ya que la información depende de las sospechas comunicadas por los notificadores y de la calidad de los datos registrados [6]. En este sentido, pueden existir problemas de infranotificación, especialmente en casos leves o no reconocidos como reacciones adversas, así como variabilidad en la completitud y precisión de la información reportada. Asimismo, la dependencia de la iniciativa del notificador introduce posibles sesgos, condicionados por factores como la experiencia clínica, el nivel de conocimiento del sistema o la percepción de gravedad del evento adverso. Adicionalmente, al no tratarse de un sistema de vigilancia activa, la capacidad para identificar de manera oportuna eventos poco frecuentes o establecer relaciones causales directas entre el medicamento y la reacción adversa puede verse limitada, requiriendo la realización de estudios complementarios más rigurosos.

1.1.3. Sistema V-safe del Centers for Disease Control and Prevention

Figura 1.5: Logotipo del sistema V-safe (vsafe.cdc.gov)

El sistema **V-safe** es una herramienta digital de monitoreo de seguridad vacunal

desarrollada por el *Centers for Disease Control and Prevention* (CDC), orientada al seguimiento de personas vacunadas en los Estados Unidos. Se lanzó originalmente en diciembre de 2020 para monitorear la seguridad de las vacunas contra el **COVID-19**. Ahora, V-safe permite rastrear a sujetos vacunados después de recibir una vacuna contra el COVID-19 o contra el **Virus Respiratorio Sincitial** (VRS)[8].

A diferencia de los sistemas tradicionales de farmacovigilancia basados en notificación espontánea, V-safe implementa un enfoque de **vigilancia activa**, en el cual el sistema contacta de manera proactiva a los usuarios mediante encuestas periódicas enviadas tras la vacunación. Después de inscribirse y seleccionar la vacuna que recibió, V-safe hace preguntas por mensajes de texto (SMS) o correo electrónico sobre cómo se siente el sujeto después de la vacunación, incluso si no presenta efectos secundarios. De este modo, el sistema está dirigido principalmente a personas vacunadas que participan de forma voluntaria, permitiendo la recopilación sistemática de información directamente desde el usuario. Recibe un chequeo diario durante la primera semana, y luego un chequeo semanal durante seis semanas adicionales [9].

La Figura 1.6 muestra un ejemplo del flujo de interacción del sistema V-safe, donde se ilustra el proceso de envío de encuestas y la recolección de información por parte de los usuarios [10].

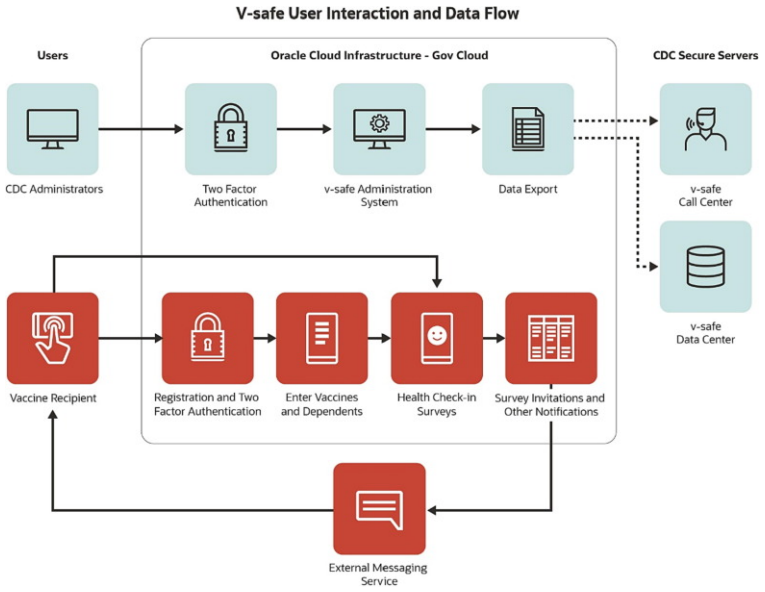


Figura 1.6: Ejemplo del flujo de interacción del sistema V-safe mediante encuestas digitales

Desde una perspectiva funcional, una de las principales fortalezas de V-safe radica en su capacidad para implementar un modelo de vigilancia activa a gran escala, permitiendo la recopilación de datos en tiempo casi real sobre la seguridad de las

vacunas. Durante su implementación inicial, el sistema demostró un alcance masivo: "más de 10 millones de personas se registraron y completaron más de 151 millones de encuestas"[10]. Este enfoque permite al CDC comprender mejor los efectos secundarios experimentados poco después de la inmunización, aportando datos que a menudo superan los obtenidos en ensayos clínicos tradicionales. El diseño del sistema facilita un seguimiento longitudinal riguroso de los individuos vacunados. Esto se logra mediante un esquema de comunicación automatizado donde el usuario recibe un çhequeo de salud diario durante la primera semana después de la vacunación y luego un chequeo semanal por seis semanas adicionales". Esta periodicidad mejora la detección temprana de posibles eventos adversos y permite una observación continuada de la salud del paciente. Asimismo, la estandarización de las encuestas y el uso de tecnologías móviles (mensajes de texto y correos electrónicos) garantizan la obtención de datos estructurados y comparables. Al emplear herramientas digitales para realizar çhequeos que típicamente toman menos de dos minutos", el sistema optimiza la recolección de información técnica que ya ha sido utilizada en más de 25 publicaciones científicas para validar la seguridad de las vacunas [10]

No obstante, el sistema presenta limitaciones asociadas a su diseño y funcionamiento. Entre ellas se encuentra la dependencia del acceso a dispositivos móviles y conectividad, lo cual puede generar sesgos en la población participante. Asimismo, al tratarse de un sistema basado en autodeclaración, la información reportada puede carecer de validación clínica directa. Adicionalmente, la participación voluntaria y longitudinal (que dura 6 semanas o más) fatiga al participante, un fenómeno documentado donde la respuesta suele decaer tras los primeros días. Finalmente, aunque el sistema permite identificar patrones y señales de seguridad, no es suficiente por sí solo para establecer relaciones causales entre la vacunación y los eventos adversos reportados.

1.1.4. Análisis comparativo de los sistemas estudiados

El análisis de los sistemas VAERS, V-safe y NotificaRAM permite identificar diferencias significativas en cuanto a su enfoque metodológico, nivel de digitalización y mecanismos de captura de información.

En primer lugar, se observa una distinción clara en el modelo de vigilancia implementado. VAERS y NotificaRAM se basan en esquemas de vigilancia pasiva, donde la calidad y cantidad de los datos dependen de la iniciativa de los notificadores. En contraste, V-safe adopta un enfoque de vigilancia activa, permitiendo la recolección sistemática y periódica de información directamente desde los usuarios vacunados.

Desde la perspectiva tecnológica, V-safe presenta un mayor grado de digitalización al integrar mecanismos automatizados de seguimiento mediante dispositivos móviles, lo que facilita la interacción continua con el usuario. Por su parte, VAERS y Notifica-

RAM, aunque disponen de plataformas web, dependen en mayor medida de la acción voluntaria del usuario para el envío de reportes.

En cuanto a los actores involucrados, los sistemas pasivos permiten una mayor diversidad de notificadores, incluyendo profesionales de la salud, fabricantes y ciudadanos. Sin embargo, esta apertura introduce variabilidad en la calidad de los datos. En contraste, V-safe se centra en usuarios previamente registrados, lo que permite una mayor consistencia en la información recolectada.

Finalmente, en términos de limitaciones, los sistemas de vigilancia pasiva presentan problemas de subregistro, sesgos de notificación y dificultades para establecer relaciones causales. Por otro lado, aunque V-safe mejora la calidad y frecuencia de los datos, su alcance se limita a los usuarios que participan voluntariamente en el sistema.

En conjunto, estos sistemas reflejan diferentes enfoques dentro de la farmacovigilancia moderna, evidenciando un tránsito desde modelos reactivos hacia estrategias más proactivas y centradas en el usuario.

1.1.5. Conclusiones sobre las soluciones actuales

El estudio de los sistemas analizados evidencia que, si bien existen soluciones consolidadas para la notificación de eventos adversos, estas presentan limitaciones significativas desde el punto de vista funcional y tecnológico.

Los sistemas basados en vigilancia pasiva, como VAERS y NotificaRAM, continúan siendo fundamentales debido a su amplia cobertura y apertura a múltiples actores. No obstante, su dependencia de la notificación voluntaria limita la completitud, oportunidad y calidad de los datos recolectados.

Por otro lado, soluciones más recientes como V-safe introducen enfoques innovadores basados en vigilancia activa y tecnologías móviles, mejorando la frecuencia y consistencia de la información. Sin embargo, estos sistemas presentan restricciones en términos de alcance poblacional y dependencia de la participación voluntaria de los usuarios.

De manera general, se identifica la necesidad de sistemas que integren las ventajas de ambos enfoques, combinando la accesibilidad y cobertura de la vigilancia pasiva con mecanismos automatizados que permitan una recolección más sistemática, oportuna y estructurada de los datos.

Estas limitaciones evidencian oportunidades para el diseño de nuevas soluciones tecnológicas orientadas a mejorar la eficiencia, calidad e interoperabilidad en la notificación de eventos adversos, lo cual constituye la base para el desarrollo del sistema propuesto en la presente investigación.

1.2. Análisis de tecnologías para la implementación del sistema propuesto

El desarrollo de sistemas informáticos modernos orientados al dominio clínico requiere la selección de un conjunto de tecnologías que garanticen propiedades fundamentales como la confiabilidad, la integridad de los datos, la escalabilidad y la mantenibilidad del software. A diferencia de aplicaciones convencionales, en este tipo de sistemas resulta especialmente relevante la correcta gestión de la información, así como la capacidad de respuesta ante un crecimiento progresivo de usuarios y volumen de datos.

En este sentido, el presente análisis se basa en el estudio de diferentes alternativas tecnológicas utilizadas en la construcción de sistemas modernos, abarcando tanto el desarrollo del backend como de las interfaces de usuario y los sistemas de gestión de bases de datos. La selección de estas tecnologías no se realiza de manera arbitraria, sino a partir de criterios técnicos previamente definidos que permiten evaluar de forma objetiva las distintas opciones disponibles.

1.2.1. Criterios técnicos para la selección de tecnologías

Con el objetivo de realizar una selección fundamentada de las tecnologías que conformarán la solución propuesta, se definen criterios de evaluación orientados a las necesidades de sistemas clínicos.

En primer lugar, se considera la **eficiencia y el rendimiento**, entendidos como la capacidad de la tecnología para responder adecuadamente bajo cargas de trabajo variables, garantizando tiempos de respuesta adecuados en escenarios de uso intensivo.

En segundo lugar, se evalúa la **escalabilidad**, referida a la capacidad del sistema para adaptarse al crecimiento en el número de usuarios y volumen de datos, tanto a nivel vertical como horizontal.

Asimismo, se analiza la **mantenibilidad**, entendida como la facilidad para realizar modificaciones, extensiones o correcciones en el software sin afectar negativamente su funcionamiento global, en función del nivel de acoplamiento y la estructura arquitectónica.

Otro criterio considerado es la **madurez tecnológica**, entendida como el nivel de estabilidad y uso comprobado de una tecnología, reflejado en su adopción en entornos reales, la calidad de su documentación y el soporte de su comunidad.

Finalmente, se introduce el criterio de **adecuación al dominio clínico**, el cual evalúa la capacidad de la tecnología para garantizar propiedades críticas como la consistencia de los datos, la trazabilidad de la información y el soporte de operaciones transaccionales seguras.

Estos criterios sirven como base para el análisis comparativo presentado en las secciones siguientes.

1.2.2. Tecnologías para el desarrollo del backend

En el desarrollo de aplicaciones web modernas, la selección del framework de backend constituye un elemento clave para la construcción del sistema. En el contexto del sistema de farmacovigilancia, se seleccionan tres tecnologías representativas del ecosistema actual: .NET, Spring Boot y NestJS. Esta selección se basa en su amplia adopción en entornos de desarrollo, su uso en aplicaciones de nivel empresarial y la diversidad de paradigmas tecnológicos que representan.

.NET Core

Figura 1.7: Logo de .NET.

.NET (anteriormente .NET Core) es un framework multiplataforma de código abierto mantenido por Microsoft, orientado al desarrollo de aplicaciones backend modernas de alto rendimiento [11].

Entre sus principales características se encuentra el soporte para la construcción de APIs REST [1] mediante ASP.NET Core. Una particularidad de ASP.NET Core es su pipeline de solicitudes HTTP, en el cual cada petición pasa por una secuencia de componentes intermedios (middlewares) que permiten procesarla, validarla o modificarla antes de generar la respuesta [12].

A diferencia de otros ecosistemas, incorpora de manera nativa un contenedor de inyección de dependencias (IServiceProvider), lo que permite que el propio framework gestione el ciclo de vida y la provisión de los componentes sin recurrir a bibliotecas de terceros [13]. Para la persistencia, utiliza Entity Framework Core, un mapeador objeto-relacional (ORM)[2] que permite trabajar con bases de datos relacionales me-

[1] API REST es una interfaz HTTP que permite operar recursos entre cliente y el servidor mediante URI y métodos estándar como GET, POST, PUT y DELETE

[2] ORM (Object-Relational Mapping) es una técnica que permite interactuar con bases de datos relacionales mediante objetos del lenguaje de programación, evitando el uso directo de consultas SQL.

diante objetos del dominio, simplificando el acceso a los datos [14].

Spring Boot

Figura 1.8: Logo de Spring Boot

Spring Boot es un framework basado en Java, de código abierto, orientado a la construcción de aplicaciones backend mediante un enfoque de configuración automática y convenciones predefinidas [15].

Entre sus principales características se encuentra el uso del paradigma de Inversión de Control (IoC), mediante el cual el framework se encarga de la creación y gestión de los componentes de la aplicación, reduciendo el acoplamiento entre ellos. Asimismo, proporciona soporte para el desarrollo de servicios web mediante Spring MVC, facilitando la implementación de APIs REST con una estructura organizada [16]. Adicionalmente, Spring Boot puede integrarse con Spring Data JPA, lo que permite la interacción con bases de datos relacionales mediante abstracciones orientadas a objetos similares a otros ORM del ecosistema.

NestJS

Figura 1.9: Logo de NestJS

NestJS es un framework de desarrollo backend basado en Node.js y TypeScript, de código abierto, utilizado para la construcción de aplicaciones del lado del servidor y APIs REST [17].

Su arquitectura se organiza mediante módulos, los cuales agrupan controladores y servicios que gestionan la lógica de la aplicación. Utiliza TypeScript como lenguaje

base, lo que permite tipado estático dentro del entorno de JavaScript. Se ejecuta sobre el modelo de Node.js basado en un bucle de eventos (event loop), lo que facilita el manejo concurrente de múltiples solicitudes mediante operaciones no bloqueantes. Además, incorpora un sistema de inyección de dependencias y puede integrarse con herramientas como TypeORM o Prisma para el acceso a bases de datos relacionales.

1.2.3. Conclusiones sobre las plataformas de desarrollo expuestas

A partir de las características técnicas descritas previamente, se realiza una comparación de las tecnologías backend seleccionadas (.NET, Spring Boot y NestJS) con base en los criterios definidos en la Sección 1.2.1.

La Tabla 1.1 resume la comparación realizada según los criterios establecidos.

Tabla 1.1: Comparación de tecnologías backend según criterios de evaluación

Criterio	**.NET**	**Spring Boot**	**NestJS**
Eficiencia	Alta (alto throughput, baja latencia, menor consumo de RAM)	Alta (rendimiento estable, mayor consumo de memoria JVM)	Media (eficiente en I/O, menor rendimiento en CPU intensivo)
Escalabilidad	Alta (pipeline ligero, alto rendimiento bajo carga y despliegue eficiente en contenedores)	Muy alta (ecosistema maduro para arquitecturas distribuidas y estabilidad en alta concurrencia)	Alta (alta concurrencia en I/O mediante modelo no bloqueante, modelo asíncrono basado en event loop)
Mantenibilidad	Alta (DI nativo, arquitectura estructurada y tipada)	Alta (IoC consolidado, ecosistema empresarial estandarizado)	Media (modularidad flexible, ecosistema heterogéneo)
Madurez	Muy alta (plataforma enterprise, soporte LTS)	Muy alta (amplia adopción empresarial consolidada)	Media (ecosistema en crecimiento)
Adecuación clínica	Alta (EF Core, transacciones ACID, seguridad integrada)	Alta (JPA/Hibernate, gestión transaccional robusta)	Media (menor adopción en sistemas clínicos críticos)

Justificación de los criterios de evaluación

A continuación, se presentan los fundamentos técnicos que sustentan la valoración asignada a cada criterio de la comparación.

Eficiencia y rendimiento La evaluación de este criterio se basa en estudios de rendimiento que analizan el comportamiento de APIs bajo escenarios de carga controlados. En este contexto, Godinho et al. (2024) reportan diferencias significativas en el consumo de memoria entre .NET y Java, donde la implementación en Java presenta aproximadamente el doble de uso de RAM en estado de reposo (*idle*), con valores de 1.40 GiB frente a 694 MiB en .NET [18].

En términos de rendimiento, los resultados de throughput indican que ASP.NET Core obtiene un mayor número de respuestas por segundo en escenarios de alta concurrencia. De forma consistente, los benchmarks de TechEmpower [3] confirman que .NET se mantiene entre las tecnologías con mejor desempeño en pruebas de múltiples consultas por solicitud, como se evidencia en la Figura 1.10 [19].

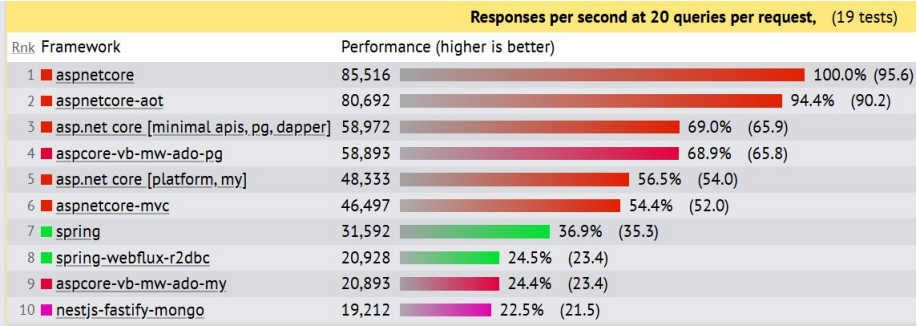


Figura 1.10: Throughput (respuestas por segundo) en .NET, Spring Boot y Node.js (adaptado de TechEmpower Framework Benchmarks)

Asimismo, los resultados de latencia evidencian que .NET presenta tiempos de respuesta más estables bajo incremento de carga.

En escenarios de estrés (*POST*), la Figura 1.11 muestra que el consumo de memoria se mantiene consistente con pruebas previas, observándose que Java requiere aproximadamente el doble de RAM que .NET, mientras que el uso de CPU presenta diferencias mínimas entre ambos frameworks [18].

Escalabilidad A partir de la comparación, se observa que tanto .NET como Spring Boot presentan un comportamiento escalable robusto en entornos distribuidos, siendo

[3] TechEmpower Framework Benchmarks es un conjunto de pruebas estandarizadas que evalúa el rendimiento de frameworks web bajo condiciones homogéneas de carga, permitiendo comparar métricas como throughput y latencia entre diferentes tecnologías como .NET, Java y Node.js.

Figura 1.11: Prueba de estrés sobre operaciones POST: uso de memoria y CPU (adaptado de *Performance Comparison of RESTful Web APIs using a Test Suite: .NET vs. Java Spring Boot* [18])

ampliamente utilizados en arquitecturas de microservicios.

Por su parte, NestJS ofrece una alta capacidad de concurrencia en escenarios ligeros gracias a su modelo de ejecución asíncrono. Sin embargo, los resultados de benchmarks indican que .NET y Spring Boot mantienen un comportamiento más consistente bajo cargas sostenidas [19].

Mantenibilidad De acuerdo con la Tabla correspondiente, .NET y Spring Boot presentan un nivel de mantenibilidad alto, principalmente debido a la consolidación de patrones arquitectónicos como IoC y la existencia de ecosistemas maduros que favorecen la estandarización del desarrollo y facilita las pruebas unitarias.

Por otro lado, NestJS presenta una mantenibilidad media, influenciada por la mayor variabilidad de su ecosistema y la dependencia de múltiples librerías externas en evolución constante.

Adicionalmente, la curva de aprendizaje también influye en la mantenibilidad del sistema, ya que tecnologías con menor complejidad inicial facilitan la incorporación de nuevos desarrolladores y reducen el tiempo de adaptación al proyecto. Esta diferencia puede observarse en la Tabla 1.2.

Madurez tecnológica .NET y Spring Boot presentan una madurez de grado empresarial consolidada, respaldada por su amplia adopción en sistemas productivos y su evolución sostenida a lo largo del tiempo, incluyendo soporte de largo plazo (LTS) en sus versiones actuales.

.NET refuerza su estabilidad mediante su ciclo de lanzamientos LTS y su fuerte integración en entornos corporativos, mientras que Spring Boot se mantiene como el estándar de facto en el ecosistema Java para aplicaciones empresariales distribuidas.

Tabla 1.2: Comparación de la curva de aprendizaje entre tecnologías backend

Tecnología	**Dificultad**	**Factores determinantes**
ASP.NET Core (.NET)	Media-Alta	Herramientas integradas (Visual Studio), arquitectura homogénea y tipado fuerte.
Spring Boot	Alta	Complejidad del ecosistema Java, uso de anotaciones y autoconfiguración.
NestJS	Media-Baja	Sintaxis accesible en TypeScript; complejidad en asincronía y concurrencia.

Por su parte, NestJS, aunque moderno y en crecimiento, presenta una madurez menor al estar basado en el ecosistema JavaScript/Node.js, caracterizado por ciclos de cambio más rápidos y menor estabilidad histórica en sistemas de misión crítica.

Adecuación al dominio clínico En sistemas de salud, la integridad, seguridad e interoperabilidad de los datos constituyen requisitos fundamentales debido a la naturaleza crítica de la información clínica.

En este contexto, .NET proporciona soporte para APIs nativas de protección de datos y estándares como *SMART on FHIR*[4], facilitando la integración con sistemas clínicos externos y el cumplimiento de normativas como HIPAA[5] [20, 21].

Asimismo, su integración con Entity Framework Core permite garantizar propiedades transaccionales de tipo ACID, asegurando consistencia, integridad y trazabilidad en operaciones críticas como el registro y auditoría de eventos adversos.

En el contexto del Instituto Finlay de Vacunas, esta madurez se evidencia en la adopción de plataformas como OpenClinica para la gestión de datos clínicos y el uso de herramientas de inteligencia de negocio como bambooBSC, ambas basadas en el ecosistema Java [22].

Spring Boot presenta una alta adecuación al dominio clínico debido a su robustez transaccional y su amplio uso en sistemas empresariales del sector salud, aunque su capa de persistencia basada en JPA/Hibernate puede incrementar la complejidad en consultas avanzadas.

[4] *SMART on FHIR* (Substitutable Medical Applications and Reusable Technologies on Fast Healthcare Interoperability Resources) es un estándar que permite el desarrollo de aplicaciones interoperables en salud mediante acceso seguro a datos clínicos estructurados sobre FHIR.

[5] HIPAA (Health Insurance Portability and Accountability Act) establece requisitos para la protección y confidencialidad de la información médica.

Por su parte, NestJS presenta una adecuación moderada, debido a su menor adopción en sistemas clínicos críticos y a la dependencia de bibliotecas externas para garantizar funcionalidades de seguridad y cumplimiento normativo.

Selección de la plataforma para el sistema de farmacovigilancia

A partir del análisis realizado, se determina que **.NET** constituye la opción más adecuada para el sistema de farmacovigilancia por las siguientes razones:

1. **Eficiencia de recursos:** Presenta un menor consumo de memoria y un alto rendimiento bajo carga en comparación con otras alternativas, lo que favorece su despliegue en entornos con recursos limitados [18].

2. **Fiabilidad en operación:** Mantiene un comportamiento estable bajo cargas sostenidas, garantizando la integridad de las transacciones críticas en el registro de eventos adversos [18].

3. **Seguridad y cumplimiento:** Dispone de mecanismos nativos para la protección de datos sensibles y cumplimiento de normativas del sector salud, lo que resulta esencial en sistemas clínicos [20].

4. **Mantenibilidad y cohesión del ecosistema:** Su arquitectura integrada, junto con herramientas unificadas y soporte de largo plazo (LTS), facilita la evolución controlada del sistema.

5. **Adecuación al contexto de desarrollo:** La experiencia previa con el ecosistema .NET reduce la curva de aprendizaje y el riesgo de implementación, favoreciendo un desarrollo más eficiente y sostenible en el tiempo.

1.2.4. Tecnologías para el desarrollo del frontend

En el desarrollo de aplicaciones web modernas, la selección de la tecnología de frontend constituye un elemento clave para garantizar la correcta implementación de interfaces de usuario eficientes, escalables y coherentes con las necesidades del sistema.

En este contexto, no se busca identificar exhaustivamente todas las alternativas existentes, sino seleccionar un conjunto representativo de tecnologías que permitan realizar un análisis comparativo basado en criterios técnicos relevantes para aplicaciones clínicas.

En este sentido, se seleccionan React, Angular y Vue por representar tres enfoques distintos en el desarrollo frontend moderno.

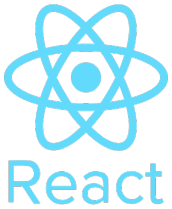


Figura 1.12: Logo de React

React

React es una biblioteca de JavaScript desarrollada por Meta para la construcción de interfaces basadas en componentes reutilizables. Su modelo declarativo utiliza un Virtual DOM para optimizar la actualización de la interfaz mediante un proceso de reconciliación [23].

Este mecanismo reduce las operaciones directas sobre el DOM real, mejorando el rendimiento en interfaces con alta frecuencia de cambios, tal como se ilustra en la Figura 1.13.

Sin embargo, React requiere la integración de librerías externas para funcionalidades como enrutamiento y gestión de estado, lo que incrementa la complejidad arquitectónica del frontend.

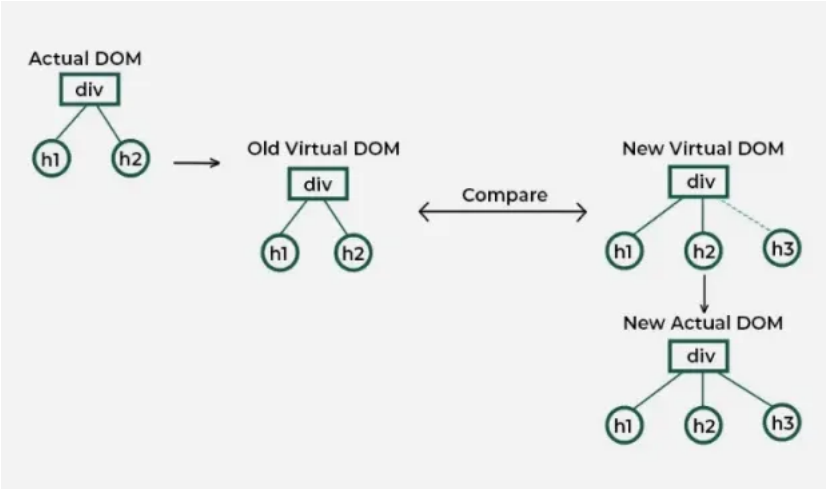


Figura 1.13: Funcionamiento del Virtual DOM en React (adaptado de *Virtual DOM in React JS*, GeeksforGeeks [24])

Figura 1.14: Logo de Angular

Angular

Angular es un framework de desarrollo frontend basado en TypeScript, mantenido por Google, orientado a la construcción de aplicaciones web de gran escala mediante una arquitectura completamente integrada [25].

Su principal característica distintiva es el uso de *two-way data binding*, el cual permite la sincronización automática entre el modelo de datos y la vista, reduciendo la necesidad de manipulación manual de la interfaz. Asimismo, su enfoque "opinionated" define una estructura de desarrollo estandarizada desde el inicio del proyecto, lo que favorece la consistencia arquitectónica en aplicaciones de gran escala.

Además, Angular proporciona una solución integral que incluye enrutamiento, inyección de dependencias, manejo de formularios y herramientas de estructuración de aplicaciones.

Vue

Figura 1.15: Logo de Vue.js

Vue.js es un framework progresivo de JavaScript diseñado para la construcción de interfaces de usuario interactivas. Su diseño incremental permite su adopción gradual, integrándose en proyectos existentes o utilizándose como framework completo [26].

Su principal característica es un sistema de reactividad que sincroniza de forma eficiente los cambios del estado con la interfaz de usuario, facilitando el desarrollo de interfaces declarativas.

Vue combina principios presentes en frameworks como React y Angular, situándose como una alternativa intermedia en términos de flexibilidad y estructura.

1.2.5. Conclusiones sobre los frameworks de frontend expuestos

Aunque los criterios definidos son aplicables de forma general al sistema, su interpretación varía según la capa tecnológica analizada.

En el frontend, la eficiencia se relaciona con el rendimiento en la actualización de la interfaz, la escalabilidad con la gestión de interfaces complejas basadas en componentes, y la adecuación al dominio clínico con la capacidad de soportar la captura, validación y visualización estructurada de datos.

Tabla 1.3: Comparación de tecnologías frontend según criterios de evaluación

Criterio	**React**	**Angular**	**Vue**
Renderizado	Alto (Virtual DOM, reconciliación eficiente)	Medio (change detection basado en zonas)	Alto (reactividad basada en dependencias)
Escalabilidad (UI)	Alta (componentes reutilizables)	Muy alta (arquitectura modular integrada)	Alta (reactividad y composición flexible)
Peso y Rendimiento	Ligero (enfocado solo en la carga de vista, agilizando el tiempo de carga inicial)	Pesado (framework pesado que incluye múltiples librerías integradas)	Muy Ligero (20-30KB ideal para cargas rápidas en redes limitadas)
Mantenibilidad	Alta (componentes + ecosistema flexible)	Muy alta (estructura integral y tipada)	Alta (estructura simple y progresiva)
Curvas de Aprendizaje	Media (ecosistema flexible con integración de librerías externas y gestión del estado)	Alta (framework integral con múltiples conceptos)	Baja-Media (enfoque progresivo y sintaxis accesible)
Madurez	Muy alta (amplia adopción y comunidad)	Muy alta (uso empresarial consolidado)	Alta (ecosistema en crecimiento)
Adecuación clínica (UI)	Alta (gestión de estado y formularios dinámicos)	Muy alta (formularios y validación integrados)	Alta (reactividad y simplicidad en formularios)

A partir de la Tabla 1.3, se identifican diferencias en los enfoques de construcción de interfaces. React y Vue priorizan mecanismos reactivos de actualización eficiente,

mientras que Angular ofrece una solución más estructurada e integrada.

Considerando los criterios evaluados, React presenta un equilibrio entre eficiencia, flexibilidad y mantenibilidad, lo cual resulta adecuado para aplicaciones basadas en componentes dinámicos.

Dado que el sistema propuesto requiere interfaces altamente interactivas orientadas a formularios clínicos, se selecciona React como tecnología base para el desarrollo del frontend.

1.2.6. Sistemas de gestión de bases de datos

En sistemas de farmacovigilancia, la gestión de datos clínicos es un elemento clave debido a la necesidad de almacenar y consultar información sensible asociada a pacientes, eventos adversos y tratamientos.

Por ello, la elección del sistema de gestión de bases de datos impacta directamente en la consistencia, integridad y escalabilidad del sistema.

En el estado del arte se consideran dos enfoques principales: bases de datos relacionales y bases de datos NoSQL, seleccionados por su amplia adopción en sistemas empresariales y por representar paradigmas complementarios de almacenamiento estructurado y flexible.

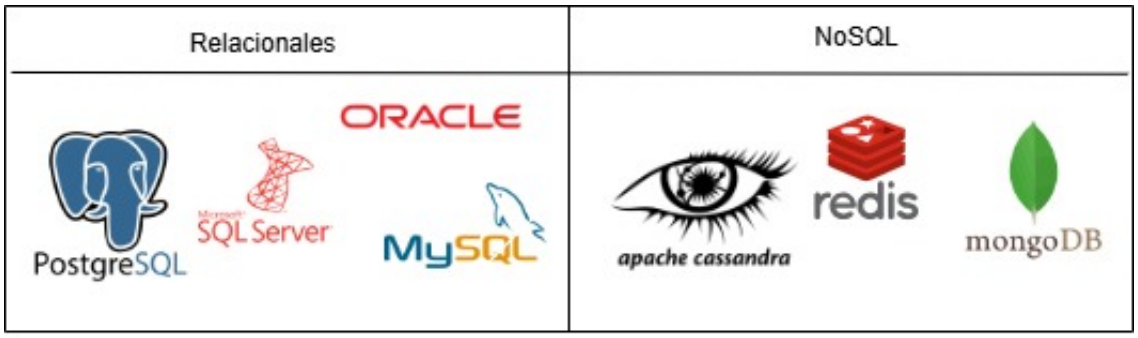


Figura 1.16: Ejemplos de sistemas de gestión de bases de datos relacionales y NoSQL

Bases de datos relacionales

Las bases de datos relacionales se basan en un modelo tabular donde la información se organiza en filas y columnas, estableciendo relaciones mediante claves primarias y foráneas, lo que permite garantizar propiedades como la integridad referencial, la consistencia de los datos y el cumplimiento de transacciones ACID [27].

No obstante, los sistemas relacionales presentan limitaciones cuando se requiere alta flexibilidad en la estructura de los datos o escalabilidad horizontal intensiva, debido a la rigidez del esquema predefinido.

Bases de datos NoSQL

Las bases de datos NoSQL surgen como alternativa a los modelos relacionales, orientadas a proporcionar mayor flexibilidad en la estructura de los datos y mejor adaptación a entornos distribuidos y de gran escala. A diferencia del modelo relacional, no requieren un esquema rígido, lo que permite gestionar datos semi-estructurados (como documentos JSON con campos variables) o no estructurados (como imágenes médicas, registros libres de texto o archivos clínicos) [28].

Sin embargo, esta flexibilidad puede comprometer la consistencia de los datos y la gestión de relaciones complejas, lo que limita su idoneidad en sistemas donde la integridad de la información es crítica, como en el dominio clínico.

1.2.7. Conclusiones sobre los sistemas de gestión de bases de datos expuestos

Se comparan dos paradigmas de bases de datos: relacionales y NoSQL, en función de criterios técnicos relevantes para sistemas clínicos.

Tabla 1.4: Comparación de paradigmas de bases de datos según criterios de evaluación

Criterio	**Relacional (SQL)**	**NoSQL**
Estabilidad	Muy alta (madurez consolidada, transacciones ACID robustas)	Alta (estable, pero dependiente del motor y configuración distribuida)
Mantenibilidad	Alta (esquema estructurado y consistente)	Media (flexibilidad del esquema puede aumentar complejidad)
Adecuación clínica	Muy alta (integridad, trazabilidad y consistencia de datos)	Media (dependiente del diseño de la aplicación)
Curva de aprendizaje	Media (modelado relacional y SQL)	Media-Baja (modelos flexibles y menos estructurados)
Escalabilidad	Media (principalmente vertical, implica el aumento de recursos en un único servidor)	Muy alta (escalabilidad horizontal nativa, implica la incorporación de múltiples nodos distribuidos)
Eficiencia	Alta en datos estructurados	Alta en grandes volúmenes de datos no estructurados

En el contexto del sistema propuesto, se selecciona MySQL como sistema de gestión de bases de datos debido a su alta consistencia, soporte transaccional y adecua-

ción al dominio clínico.

1.2.8. Síntesis comparativa de las tecnologías analizadas

A partir del análisis realizado en las secciones anteriores, se evidencia que las tecnologías evaluadas presentan características diferenciadas en función del componente del sistema al que están orientadas, así como de los paradigmas de diseño que promueven.

En el ámbito del backend, se identifican soluciones orientadas a la construcción de sistemas robustos y mantenibles, destacándose plataformas consolidadas como .NET y Spring Boot por su madurez, soporte arquitectónico y enfoque en aplicaciones empresariales, así como alternativas más recientes como NestJS, que introducen modelos modulares y flexibles dentro del ecosistema JavaScript.

En cuanto al desarrollo de interfaces de usuario, se observa una tendencia hacia arquitecturas basadas en componentes y modelos declarativos, donde tecnologías como React, Angular y Vue ofrecen distintos niveles de estructuración, flexibilidad y control sobre la complejidad de la aplicación, en función de los requerimientos de interacción y dinamismo de la interfaz.

Por otra parte, en el contexto de la persistencia de datos, las tecnologías analizadas reflejan enfoques diferenciados en cuanto al modelado y gestión de la información, lo cual influye directamente en aspectos como la consistencia, la integridad y la adaptabilidad del sistema ante cambios en los datos.

De manera integral, se evidencia que la selección tecnológica no debe realizarse de forma aislada por componente, sino considerando la coherencia entre las distintas capas del sistema. En particular, la combinación de tecnologías backend, frontend y de persistencia debe garantizar un equilibrio entre rendimiento, mantenibilidad y confiabilidad, aspectos especialmente críticos en sistemas orientados a la gestión de información clínica.

Estos elementos constituyen la base para la selección de las tecnologías que serán empleadas en la implementación del sistema, la cual se aborda en el capítulo siguiente.

1.3. Tecnologías y enfoques para la gestión de datos clínicos

La gestión de datos clínicos en sistemas de farmacovigilancia constituye un problema de naturaleza compleja, en el cual convergen requisitos de interoperabilidad semántica, consistencia estructural, trazabilidad de la información y calidad en la captura de los datos. A diferencia de los sistemas de información convencionales, los entornos clínicos demandan un tratamiento riguroso de la información debido a su

impacto directo en procesos de análisis sanitario, toma de decisiones en salud pública y evaluación de la seguridad de las vacunas.

En este contexto, los enfoques tecnológicos asociados a la gestión de datos clínicos no se limitan únicamente a herramientas de almacenamiento o procesamiento, sino que abarcan también estándares de representación de la información, modelos de captura de datos y mecanismos de validación que permiten garantizar la integridad de los registros desde su origen hasta su utilización en procesos analíticos.

Asimismo, la heterogeneidad de las fuentes de datos y la variabilidad en la forma en que los usuarios reportan eventos adversos introduce el denominado problema de "ruido informativo", el cual dificulta la integración y el análisis automatizado de la información clínica. Por esta razón, resulta necesario el uso de estándares y enfoques que permitan normalizar la representación de los datos, así como establecer reglas de validación que aseguren su coherencia y fiabilidad.

De igual forma, los sistemas de farmacovigilancia modernos han evolucionado hacia modelos que combinan distintos paradigmas de captura de información, incluyendo mecanismos de reporte en línea y modelos de captura diferida mediante formularios estructurados, con el objetivo de garantizar la disponibilidad del sistema en escenarios con conectividad limitada sin comprometer la calidad de los datos recolectados.

En las subsecciones siguientes se analizan los principales estándares internacionales utilizados en la representación de información clínica, los modelos de captura de datos empleados en sistemas de autorreporte de eventos adversos, así como los enfoques de validación de datos clínicos, con el objetivo de fundamentar las decisiones de diseño adoptadas en la presente investigación.

1.3.1. Estándares de interoperabilidad clínica

La interoperabilidad en sistemas de información en salud constituye un requisito esencial para garantizar que los datos clínicos puedan ser compartidos, interpretados y reutilizados de manera consistente entre diferentes sistemas y actores del ecosistema sanitario. En el contexto de la farmacovigilancia de vacunas, esta capacidad resulta fundamental para asegurar que la información reportada mantenga su significado original y pueda ser utilizada de forma fiable en procesos de análisis y toma de decisiones.

Uno de los estándares más relevantes en este ámbito es HL7 FHIR (Fast Healthcare Interoperability Resources), desarrollado por la organización HL7 [29], el cual define un modelo basado en recursos que permite estructurar la información clínica de manera modular e interoperable. Este estándar facilita la representación de entidades como pacientes, vacunas administradas y eventos clínicos asociados, permitiendo la integración entre sistemas heterogéneos.

El modelo basado en FHIR introduce una estructura flexible donde cada recur-

so representa una unidad de información clínica que puede ser combinada con otras para describir eventos complejos. En el caso de los eventos adversos asociados a vacunas, esta estructura permite relacionar el paciente, la inmunización recibida y las observaciones clínicas derivadas.

Por otra parte, la estandarización terminológica constituye un elemento clave para evitar ambigüedades en la interpretación de los datos. En este sentido, el diccionario MedDRA (Medical Dictionary for Regulatory Activities) proporciona una terminología controlada para la codificación de eventos adversos [30], permitiendo la representación uniforme de síntomas y reacciones asociadas a vacunas.

Como se observa en la Figura 1.17, MedDRA organiza los términos clínicos en una estructura jerárquica de cinco niveles, que permite representar un mismo concepto clínico con distintos grados de especificidad. Esta jerarquía va desde categorías generales (System Organ Class) hasta términos altamente específicos (Lowest Level Term), lo cual facilita la normalización semántica de los eventos reportados y reduce la ambigüedad en la interpretación de los datos clínicos.

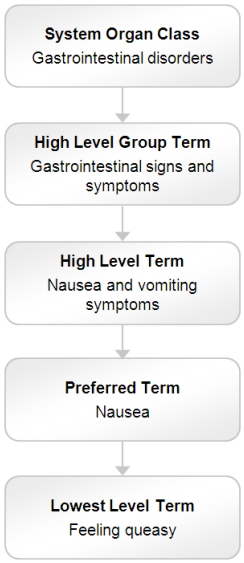


Figura 1.17: Estructura jerárquica del sistema de codificación MedDRA.

De manera complementaria, la Clasificación Internacional de Enfermedades (CIE), desarrollada por la Organización Mundial de la Salud [31], proporciona un marco estandarizado para la codificación de condiciones de salud, lo cual permite enriquecer el contexto clínico de los reportes cuando es necesario.

En conjunto, la adopción de estándares de interoperabilidad y terminologías controladas como FHIR, MedDRA y CIE permite mejorar la calidad, consistencia y utilidad de los datos clínicos, constituyendo un elemento fundamental en el diseño de

sistemas orientados al reporte estructurado de eventos adversos asociados a vacunas.

1.3.2. Modelos de captura de datos clínicos

La captura de datos clínicos constituye una etapa crítica en el diseño de sistemas de información en salud, ya que determina en gran medida la calidad, consistencia y utilidad de la información registrada. En el contexto de los sistemas de reporte de eventos adversos asociados a vacunas, una captura inadecuada puede derivar en pérdidas de información relevante, ambigüedades en los datos o dificultades en su posterior procesamiento.

En la práctica, pueden identificarse diferentes modelos de captura de datos clínicos en función del nivel de estructuración de la información desde su origen.

El modelo de captura estructurada se basa en la utilización de formularios con campos previamente definidos, lo cual permite restringir los valores de entrada y facilitar la validación automática de los datos. Este enfoque es ampliamente utilizado en sistemas de información en salud modernos, debido a su capacidad para mejorar la calidad del dato desde su origen y facilitar su procesamiento posterior.

El modelo semi-estructurado constituye un enfoque intermedio, donde la información se organiza bajo una plantilla definida pero permite cierto grado de flexibilidad en su completitud. Un ejemplo representativo de este enfoque es el formulario en formato PDF utilizado en sistemas de farmacovigilancia como el Vaccine Adverse Event Reporting System (VAERS), el cual permite la recolección de información incluso en escenarios donde no existe conectividad continua o acceso a plataformas en línea [1].

Finalmente, el modelo no estructurado se basa en la captura de información en forma de texto libre o descripciones narrativas. Aunque este enfoque puede aportar mayor riqueza descriptiva, presenta limitaciones significativas en términos de procesamiento automático, normalización de datos e integración con sistemas de análisis clínico.

Desde la perspectiva del diseño de sistemas de información, los modelos estructurados y semi-estructurados resultan más adecuados para escenarios de farmacovigilancia, ya que permiten garantizar un mayor control sobre la calidad de los datos desde su origen, facilitando su integración con procesos posteriores de validación, normalización y análisis.

En consecuencia, la selección del modelo de captura debe equilibrar la precisión de la información, la facilidad de uso para el usuario final y la capacidad de integración con los procesos de análisis del sistema.

1.3.3. Herramientas para la validación reactiva de formularios

La validación de datos en tiempo de captura constituye un mecanismo esencial para garantizar la calidad y consistencia de la información en sistemas de autorreporte clínico. En este contexto, la validación reactiva se refiere al proceso de verificación de los datos introducidos por el usuario de manera inmediata, es decir, antes de su envío al servidor, permitiendo detectar y corregir errores en etapas tempranas del flujo de interacción.

Este tipo de validación resulta especialmente relevante en sistemas de reporte de eventos adversos asociados a vacunas, donde la precisión de los datos ingresados impacta directamente en la confiabilidad del análisis posterior.

Las técnicas de validación pueden clasificarse en validaciones sintácticas, orientadas a verificar el formato de los datos (como fechas, números o identificadores), y validaciones semánticas básicas, enfocadas en garantizar coherencia lógica entre los campos del formulario. Ejemplos de estas incluyen la verificación de campos obligatorios, rangos válidos de valores o consistencia entre fechas relacionadas.

En el desarrollo de aplicaciones modernas, existen múltiples herramientas que facilitan la implementación de validación reactiva. En el ecosistema frontend basado en React, bibliotecas como React Hook Form y Formik permiten gestionar formularios complejos con validación en tiempo real [32] [33]. De manera similar, en Angular se emplean formularios reactivos que integran validación mediante el módulo Reactive Forms [34].

En el ámbito del backend, frameworks como .NET incorporan bibliotecas como FluentValidation, que permiten definir reglas de validación de forma declarativa y reutilizable [35]. En ecosistemas basados en Node.js, bibliotecas como Joi o Zod proporcionan mecanismos de validación estructurada de datos antes de su persistencia.

La utilización de estas herramientas permite reducir significativamente la probabilidad de inserción de datos inconsistentes, mejorando la calidad de la información desde su punto de origen y complementando los procesos posteriores de normalización e integración en el sistema.

1.3.4. Calidad e integración de datos clínicos

La calidad de los datos clínicos constituye un factor determinante en la utilidad de los sistemas de información en salud, especialmente en contextos donde la información es utilizada para el análisis de eventos adversos asociados a vacunas. En este tipo de sistemas, la presencia de datos incompletos, inconsistentes o ambiguos puede comprometer significativamente la fiabilidad de los resultados obtenidos.

Desde una perspectiva de ingeniería de software, la calidad de los datos puede

evaluarse a partir de múltiples dimensiones, entre las que se incluyen la integridad, la consistencia, la exactitud, la completitud y la trazabilidad. Estas propiedades garantizan que la información almacenada represente de manera fiel los eventos del mundo real y pueda ser utilizada de forma confiable en procesos posteriores de análisis.

La integración de datos clínicos se refiere al proceso mediante el cual la información capturada y validada en el sistema es unificada bajo un modelo común, permitiendo su interoperabilidad y su posterior procesamiento. En este sentido, la utilización de un modelo de datos estructurado resulta esencial para evitar la fragmentación de la información y facilitar su análisis global.

En el contexto de los sistemas de autorreporte de eventos adversos, la integración de datos se apoya en el uso de estándares de interoperabilidad como HL7 FHIR [29], que permite la representación estructurada de entidades clínicas, y en terminologías controladas como MedDRA [30] y la Clasificación Internacional de Enfermedades (CIE) de la Organización Mundial de la Salud [31], que aseguran la normalización semántica de los conceptos médicos.

Estos estándares permiten establecer un lenguaje común entre sistemas heterogéneos, facilitando la interoperabilidad, la coherencia semántica de los datos y su reutilización en diferentes contextos clínicos.

Asimismo, la integración adecuada de los datos clínicos permite establecer relaciones coherentes entre las distintas entidades del sistema, como el paciente, la vacunación administrada y las reacciones adversas reportadas, garantizando la trazabilidad completa del evento clínico.

En conjunto, la aplicación de principios de calidad de datos y mecanismos de integración estructurada permite mejorar la confiabilidad del sistema, reducir la ambigüedad en la interpretación de la información y facilitar su uso en procesos de análisis clínico y toma de decisiones.

1.4. Conclusiones del capítulo

A partir del análisis realizado en el presente capítulo, se evidencia que los sistemas de autorreporte de eventos adversos asociados a vacunas constituyen herramientas fundamentales dentro de los procesos de farmacovigilancia moderna. Sin embargo, los sistemas estudiados como VAERS, NotificaRAM y V-safe presentan limitaciones relacionadas con la heterogeneidad en la captura de datos, la dependencia de estructuras de reporte específicas y, en algunos casos, restricciones en la validación temprana de la información reportada.

En cuanto a las tecnologías utilizadas para la implementación de sistemas de información en este dominio, se observa la existencia de múltiples alternativas consolidadas para el desarrollo del backend, frontend y la gestión de bases de datos. No obstante,

la selección de dichas tecnologías debe responder a criterios de robustez, escalabilidad, mantenibilidad y, especialmente, a la capacidad de garantizar la integridad de los datos en contextos clínicos.

Asimismo, el análisis de los modelos de captura de datos clínicos evidencia que los enfoques estructurados y semi-estructurados constituyen las alternativas más adecuadas para sistemas de autorreporte, debido a su capacidad para reducir la ambigüedad en la información y facilitar su posterior procesamiento. En particular, los modelos basados en formularios estructurados y formatos interoperables como los utilizados en sistemas de referencia permiten mejorar la calidad del dato desde su origen.

Por otra parte, la incorporación de herramientas de validación reactiva en el punto de captura representa un elemento esencial para la reducción de errores en la entrada de datos, contribuyendo directamente a la consistencia de la información almacenada. De forma complementaria, los mecanismos de integración y calidad de datos clínicos, apoyados en estándares como HL7 FHIR, MedDRA y la Clasificación Internacional de Enfermedades (CIE), permiten establecer un marco común para la representación y normalización de la información clínica.

En conjunto, el estado del arte analizado permite identificar que la principal problemática en los sistemas actuales no radica únicamente en la disponibilidad de tecnologías, sino en la integración coherente de modelos de captura, validación, estandarización e interoperabilidad de datos clínicos. Esta situación fundamenta la necesidad de proponer una solución que unifique estos elementos dentro de un modelo computacional consistente orientado al reporte estructurado de eventos adversos asociados a vacunas.

Capítulo 2

Propuesta

Capítulo 3

Detalles de Implementación y Experimentos

Conclusiones

Conclusiones

Recomendaciones

Recomendaciones

Bibliografía

[1] Vaccine Adverse Event Reporting System. *Vaccine Adverse Event Reporting System (VAERS)*. Accedido: 06 de abril de 2026. 2024. URL: https://vaers.hhs.gov/ (vid. págs. 1, 2, 25).

[2] Centers for Disease Control and Prevention. *About the Vaccine Adverse Event Reporting System (VAERS)*. Accedido: 06 de abril de 2026. 2024. URL: https://www.cdc.gov/vaccine-safety-systems/vaers/index.html (vid. págs. 1, 2, 4).

[3] U.S. Food and Drug Administration. *VAERS Overview*. Accedido: 06 de abril de 2026. 2024. URL: https://www.fda.gov/vaccines-blood-biologics/vaccine-adverse-events/vaers-overview (vid. págs. 2, 4).

[4] Agencia Española de Medicamentos y Productos Sanitarios. *Manual de uso NotificaRAM*. Accedido: 06 de abril de 2026. 2025. URL: https://www.notificaram.es (vid. págs. 4, 5).

[5] Agencia Española de Medicamentos y Productos Sanitarios. *Sistema Español de Farmacovigilancia de Medicamentos de Uso Humano*. Accedido: 06 de abril de 2026. 2025. URL: https://www.aemps.gob.es/medicamentos-de-uso-humano/farmacovigilancia-de-medicamentos-de-uso-humano/identificacion-de-riesgos/sistema-espanol-de-farmacovigilancia-de-medicamentos-de-uso-humano/ (vid. págs. 4, 5).

[6] Agencia Española de Medicamentos y Productos Sanitarios. *Notificación de sospechas de reacciones adversas a medicamentos (RAM) de uso humano*. Accedido: 06 de abril de 2026. 2026. URL: https://www.aemps.gob.es/medicamentos-de-uso-humano/farmacovigilancia-de-medicamentos-de-uso-humano/notificacion-de-sospechas-de-reacciones-adversas-a-medicamentos-ram-de-uso-humano/ (vid. págs. 4, 5).

[7] Agencia Española de Medicamentos y Productos Sanitarios. *Información sobre el acceso a los datos de FEDRA*. Accedido: 06 de abril de 2026. 2026. URL: https://www.aemps.gob.es/medicamentos-de-uso-humano/farmacovigilancia-

de-medicamentos-de-uso-humano/informacion-sobre-el-acceso-a-los-datos-de-fedra/ (vid. pág. 5).

[8] Centers for Disease Control and Prevention. *About V-safe*. Accedido: 06 de abril de 2026. 2025. URL: https://www.cdc.gov/vaccine-safety-systems/v-safe/index.html (vid. pág. 6).

[9] Centers for Disease Control and Prevention. *V-safe After Vaccination Health Checker*. Accedido: 06 de abril de 2026. 2024. URL: https://vsafe.cdc.gov (vid. pág. 6).

[10] Tanya R. Myers et al. «The v-safe after vaccination health checker: Active vaccine safety monitoring during CDC's COVID-19 pandemic response». En: *Vaccine* 41.7 (2023). Disponible en PubMed Central, págs. 1310-1318. DOI: 10.1016/j.vaccine.2022.12.031 (vid. págs. 6, 7).

[11] Microsoft. *.NET Documentation*. https://dotnet.microsoft.com/en-us/. Accessed: 2026-04-10. 2024 (vid. pág. 10).

[12] Microsoft Developer Division. *Architect Modern Web Applications with ASP.NET Core and Azure*. Accessed: 2026-04-10. Microsoft, 2023. URL: https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/ (vid. pág. 10).

[13] Kirk Larkin, Steve Smith y Brandon Dahler. *Dependency injection in ASP.NET Core*. 2026. URL: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection (vid. pág. 10).

[14] Wikipedia contributors. *Entity Framework Core*. 2026. URL: https://en.wikipedia.org/wiki/Entity_Framework (vid. pág. 10).

[15] VMware, Inc. *Spring Boot Documentation*. https://spring.io/projects/spring-boot. Accessed: 2026-04-10. 2024 (vid. pág. 10).

[16] Craig Walls. *Spring in Action*. 5th. Manning Publications, 2019 (vid. pág. 11).

[17] NestJS Team. *NestJS Documentation*. https://docs.nestjs.com/. Accessed: 2026-04-10. 2024 (vid. pág. 11).

[18] Antonio Godinho et al. «Performance Comparison of RESTful Web APIs using a Test Suite: .NET vs. Java Spring Boot». En: *Journal of Software & Systems Development* (2024). DOI: 10.5171/2024.478010 (vid. págs. 13, 14, 16).

[19] TechEmpower. *Web Framework Benchmarks*. https://www.techempower.com/benchmarks/. Accessed: April 20, 2026. 2025 (vid. pág. 13).

[20] Rushil Bhuptani. *Building HIPAA-Compliant Healthcare Apps with .NET: Best Practices and Pitfalls*. Publicado originalmente en febrero de 2025 y modificado el 10 de marzo de 2026. Mar. de 2026. URL: https://www.avidclan.com/blog/building-hipaa-compliant-healthcare-apps-with-dot-net-best-practices-and-pitfalls/ (vid. págs. 15, 16).

[21] Darena Health. *Build a SMART on FHIR App in ASP.NET Core (C#) from Scratch with MeldRx*. Video tutorial sobre la implementación de estándares HL7 FHIR en el ecosistema .NET. YouTube. 2024. URL: https://youtu.be/UKTisHLeqn4?si=wy_h-HnsJKoBFYya (vid. pág. 15).

[22] Ing. Judith de las Mercedes Carbonell Alfonso. «Propuesta de diseño del Cuadro de Mando Integral para la toma de decisiones en el Instituto Finlay de Vacunas». Tutor: Dr. Luis del Castillo Sánchez. Tesis en opción al grado de Máster en Gestión de Información. La Habana, Cuba: Universidad de La Habana, Facultad de Economía, 2016 (vid. pág. 15).

[23] Meta Open Source. *React Documentation*. https://react.dev. Accessed: 2026-04-11. 2024 (vid. pág. 17).

[24] GeeksforGeeks. *ReactJS Virtual DOM*. Accedido en abril de 2026. 2026. URL: https://www.geeksforgeeks.org/reactjs/reactjs-virtual-dom/ (vid. pág. 17).

[25] Google. *Angular Documentation*. https://angular.io. Accessed: 2026-04-11. 2024 (vid. pág. 18).

[26] Vue.js Team. *Vue.js Guide*. https://vuejs.org. Accessed: 2026-04-11. 2024 (vid. pág. 18).

[27] Ramez Elmasri y Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson, 2016 (vid. pág. 20).

[28] ABM Moniruzzaman y Syed Akhter Hossain. «Nosql database: New era of databases for big data analytics-classification, characteristics and comparison». En: *arXiv preprint arXiv:1307.0191* (2013) (vid. pág. 21).

[29] HL7 International. *FHIR Overview*. Accedido: 2026-04-11. 2024. URL: https://www.hl7.org/fhir/overview.html (vid. págs. 24, 27).

[30] International Council for Harmonisation (ICH). *Medical Dictionary for Regulatory Activities (MedDRA)*. Accedido: 2026-04-11. 2024. URL: https://www.meddra.org/ (vid. págs. 24, 27).

[31] World Health Organization. *International Classification of Diseases (ICD)*. Accedido: 2026-04-11. 2024. URL: https://www.who.int/standards/classifications/classification-of-diseases (vid. págs. 25, 27).

[32] *React Hook Form Documentation*. https://react-hook-form.com/. Accedido: 2026-04-11 (vid. pág. 26).

[33] *Formik Documentation*. https://formik.org/. Accedido: 2026-04-11 (vid. pág. 26).

[34] *Angular Reactive Forms Guide*. https://angular.io/guide/reactive-forms. Accedido: 2026-04-11 (vid. pág. 26).

[35] *FluentValidation Documentation*. https://docs.fluentvalidation.net/. Accedido: 2026-04-11 (vid. pág. 26).